

COMP 3311 DATABASE MANAGEMENT SYSTEMS

MIDTERM PRACTICE TEST

Coverage: Lectures 1–11 | Database Design · Relational Design · Normalization · Relational Algebra · SQL

TOPICS COVERED

- E-R Data Model (entities, attributes, relationships, constraints, weak entities, generalization)
 - Reduction of E-R Schemas to Relational Schemas
 - Functional Dependencies (closure, canonical cover, candidate keys)
 - Normalization (1NF, 2NF, 3NF, BCNF — decomposition algorithms)
 - Relational Algebra (basic + additional operations)
 - SQL DML (SELECT, JOIN, aggregation, subqueries, analytic functions, data modification)
 - SQL DDL (CREATE TABLE, constraints, views, triggers, PL/SQL)
-

REFERENCE SCHEMAS

University Schema

```
Student(studentId, name, year, deptName)
Course(courseNo, title, credits, deptName)
Instructor(instId, name, deptName, salary)
Enrol(studentId, courseNo, semester, grade)
Teaching(instId, courseNo, semester, room)
Department(deptName, building, budget)
```

Bank Schema

```
Account(accountNo, balance, branchName)
Borrower(clientId, loanNo)
Branch(branchName, district, liabilities, assets)
Client(clientId, name, hkid, address, district, rating)
Depositor(clientId, accountNo)
```

```
Loan(loanNo, amount, year, branchName)
Tags(clientId, tag)
```

QUESTION 1 — E-R Diagram Design

A hospital manages its wards, doctors, and patients. The requirements are:

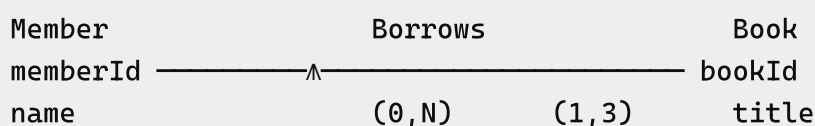
- Each **Ward** is identified by a unique ward number and has a name and floor number.
- Each **Doctor** is identified by a unique doctor ID and has a name, specialty, and phone number. A doctor's phone number may have multiple values.
- Each **Patient** is identified by a unique patient ID and has a name, date of birth, and age. Age is computed from date of birth.
- Every patient is assigned to exactly one ward. A ward may have many patients or none at all (e.g., a newly built ward).
- A doctor treats many patients, and a patient may be treated by many doctors. For each treatment, the hospital records the treatment date and diagnosis.
- A doctor must treat at least one patient.
- The hospital distinguishes between two types of doctors: **Surgeons** (who have a specialization area) and **Physicians** (who have an office consultation fee). Every doctor is either a surgeon or a physician, but not both.

Draw a complete E-R diagram showing all entity types, attributes, relationships, relationship constraints (cardinality and participation), weak entities (if any), and generalization/specialization. Use proper Crow's Foot or UML notation.

QUESTION 2 — E-R to Relational Schema Reduction

Below is an outline of the reduction of a library E-R schema to relation schemas. **Complete the reduction** for each relation schema by:

1. Adding any required additional attributes
2. Underlining the primary key
3. Writing the referential integrity constraints (foreign keys with ON DELETE actions) below each schema
4. Minimizing the number of relation schemas



address	dateDue	publisher
phone		year
{mobile}		

Book $\xrightarrow{(1,N)}$ WrittenBy $\xrightarrow{(1,1)}$ Author
 authorId
 name

Additional notes:

- Member's {mobile} is a multi-valued attribute.
- Member(memberId, name, address, phone) — **already partially reduced** (mobile not yet handled).
- A book must have at least one author (total participation from Book in WrittenBy).
- A member may borrow 0 to 3 books at a time; each book may be borrowed by 0 to N members.
- dateDue records the due date for each borrow instance.

Complete the reduction:

Member(_____)

Book(_____)

Author(_____)

Borrows(_____)

WrittenBy(_____)

QUESTION 3 — Reverse Engineering E-R from Relations

Given the following relation schemas (keys underlined, foreign keys in *italics*):

```
Employee(eid, name, salary, *deptCode*)
Department(deptCode, deptName, location, *mgrEid*)
Project(projNo, title, budget, *deptCode*)
WorksOn(*eid*, *projNo*, hours)
Dependent(*eid*, depName, birthDate, relationship)
```

Additional semantics:

- `Employee.deptCode` references `Department(deptCode)`. Every employee must belong to a department.
- `Department.mgrEid` references `Employee(eid)`. Every department must have a manager.
- `Project.deptCode` references `Department(deptCode)`. Every project is owned by exactly one department.
- `WorksOn` links employees to projects with the hours they worked. An employee may work on many projects, and a project may involve many employees.
- `Dependent` is a weak entity identified by `eid` (from `Employee`) plus `depName`. An employee may have zero or more dependents.

3.1 CREATE TABLE Order

Given the foreign keys above and assuming all referential integrity constraints are included in the SQL `CREATE TABLE` statements, **what should be the creation order?** Explain your answer.

3.2 Construct an E-R Schema

Construct an E-R schema that reduces to the given relation schemas. Your E-R schema should minimize the number of entities and relationships. Clearly show relationship constraints (cardinality and participation), weak entities, and identifying relationships.

QUESTION 4 — Functional Dependencies & Normalization

Given: $R(A, B, C, D, E, F)$

$F = \{ AB \rightarrow C, C \rightarrow D, AB \rightarrow E, E \rightarrow F, D \rightarrow A \}$

4.1 Attribute Closure

What is $\{A, B\}^+$ (the closure of $\{A, B\}$ under F)? Show your working step by step.

4.2 Candidate Keys

Which of the following is a **candidate key** for R ?

- a) AB
- b) ABC
- c) ABE
- d) ABD
- e) More than one of the above

Show your reasoning for each option.

4.3 Prime Attributes

List all **prime attributes** of R . Explain your reasoning.

4.4 Normalization — 3NF Check

Is R in **3NF**? If not, identify a violating FD and explain why it violates 3NF.

Recall: R is in 3NF iff for each FD $X \rightarrow A$ in F^+ :

- $A \in X$ (trivial FD), or
- X is a superkey for R , or
- A is a prime attribute for R .

4.5 3NF Decomposition

Decompose R into **3NF** using the synthesis algorithm. Show:

1. The canonical cover F_c
2. Each relation schema in the decomposition
3. Whether the decomposition is lossless-join and dependency-preserving

4.6 BCNF Check

Is R in **BCNF**? If not, identify a violating FD and explain why.

4.7 BCNF Decomposition

Decompose R into **BCNF**. Show each step. Is your final decomposition dependency-preserving? If not, which FD(s) are lost?

4.8 3NF vs BCNF — Trade-off

For this specific R and F, which would you recommend: the 3NF decomposition or the BCNF decomposition? Justify your answer in 2–3 sentences.

QUESTION 5 — Relational Algebra

Use only the University Schema (see Reference Schemas at the top of this test).

5.1

Construct a **single relational algebra expression** to find the **names of students** who have enrolled in a course titled "Database Systems".

5.2

Construct a **single relational algebra expression** to find the **names of instructors** who have taught at least one course in the "Computer Science" department but have **never** taught a course in the "Mathematics" department.

5.3

Construct a **single relational algebra expression** that produces the same result as the following SQL query:

```
SELECT deptName, COUNT(*)  
FROM Student  
GROUP BY deptName  
HAVING COUNT(*) >= 5;
```

Hint: SQL GROUP BY with HAVING cannot be directly expressed in basic relational algebra. Use renaming, Cartesian product, and set operations to simulate counting and filtering.

5.4

Construct a **single relational algebra expression** to find the **names and salaries of instructors** who earn more than **every** instructor in the "History" department.

QUESTION 6 — SQL Queries

Use only Oracle SQL constructs presented in the lectures. Use the Bank Schema and University Schema as indicated.

6.1 Basic Query (Bank Schema)

Construct a **single SQL query** to find the `clientId` and `name` of clients who have **both** a loan and an account at the "Star House" branch. Do **not** use subqueries.

6.2 Aggregation & HAVING (Bank Schema)

Construct a **single SQL query** to find the `branchName` and total number of loans for branches that have issued **more than 3 loans**. Order the result by total loans descending.

6.3 Subquery with NOT EXISTS (Bank Schema)

Construct a **single SQL query** using `NOT EXISTS` to find the `clientId` and `name` of clients who have accounts at **every** branch in the "Yau Tsim Mong" district.

6.4 WITH Clause / CTE (University Schema)

Construct a **single SQL query** using a `WITH` clause to find the `deptName` and `name` of the **highest-paid instructor** in each department. If there is a tie, include all tied instructors.

6.5 Analytic Functions — Top-N (Bank Schema)

Construct a **single SQL query** using an analytic function to find the **top 3 branches** by total assets. If there is a tie, include all tied branches. Use `RANK()` or `DENSE_RANK()` and explain your choice.

6.6 LISTAGG (University Schema)

Construct a **single SQL query** to find, for each department, the department name, its total budget, and a **comma-separated list of student names** enrolled in that department, ordered alphabetically by student name. Only include departments with more than 2 students.

6.7 Equivalence Rewrite (Bank Schema)

Rewrite the following query **without using** `SOME` or `ALL` :

```
SELECT branchName
FROM Branch
WHERE assets > ALL (SELECT assets FROM Branch WHERE district = 'Yau Tsim
Mong');
```

6.8 Data Modification (Bank Schema)

Write SQL statements to:

1. **Insert** a new client with `clientId = 'C-999'`, `name = 'Chan Tai Man'`, `hkid = 'A123456(7)'`, `address = '1 Harbour Road'`, `district = 'Wan Chai'`, and `rating = 7`.
2. **Update** all accounts at the "Pacific Place" branch: increase the balance by 6% for accounts with balance $\leq 10,000$, and by 4% for all other accounts. Use a **single UPDATE statement** with `CASE`.
3. **Delete** all borrowers whose loans are at branches in the "Yau Tsim Mong" district.

QUESTION 7 — SQL DDL & PL/SQL

Use Oracle SQL constructs. Use the Bank Schema for reference.

7.1 CREATE TABLE

Write a complete `CREATE TABLE` statement for the `Loan` relation with the following specifications:

- `loanNo` : `CHAR(5)`, primary key
- `amount` : `NUMBER(8,2)`, must be between 1000 and 200000
- `year` : `CHAR(4)`, cannot be null
- `branchName` : `VARCHAR2(15)`, cannot be null, foreign key referencing `Branch(branchName)`
- When a branch is deleted, all its loans should be deleted automatically.

7.2 View

Create a view called `HighValueClients` that shows the `clientId`, `name`, and `district` of all clients who have a rating greater than or equal to 8. Then write a query that uses this view to count how many high-value clients exist in each district.

7.3 PL/SQL Procedure

Write a PL/SQL procedure called `TransferFunds` that:

- Takes two account numbers (`fromAccount` and `toAccount`) and an `amount`
- Transfers the `amount` from `fromAccount` to `toAccount`
- Raises an exception if `fromAccount` has insufficient balance
- Handles the case where either account does not exist

Include the complete procedure definition with exception handling.

7.4 Trigger

Write a trigger `update_branch_liabilities` that fires **after** a new loan is inserted into the `Loan` table. The trigger should automatically update the `liabilities` of the corresponding branch by adding the new loan amount to the branch's current liabilities.

QUESTION 8 — Short Answer

8.1

Explain the difference between **logical data independence** and **physical data independence** in the three-level DBMS architecture. Give a concrete example of each.

8.2

Explain the difference between `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()` in Oracle SQL. Provide a small example where all three produce different results.

8.3

What is the difference between **3NF** and **BCNF**? Why might a designer choose 3NF over BCNF in practice?

8.4

Explain what *horror vacui* means in the context of cartography. Just kidding — explain the difference between **aggregate functions** and **analytic functions** in Oracle SQL. When would you use one over the other?

8.5

Explain the difference between **WHERE** and **HAVING** in SQL. In what order are they evaluated, and why does this matter?

ANSWER KEY

QUESTION 1 — E-R Diagram (Solution Notes)

The E-R diagram should include:

Entities:

- **Ward** (strong entity): PK = wardNo, attributes: name, floorNo
- **Doctor** (strong entity): PK = docId, attributes: name, specialty, {phone} (multi-valued)
- **Patient** (strong entity): PK = patientId, attributes: name, dob, (age) (derived)
- **Surgeon** (subtype of Doctor): attribute: specializationArea
- **Physician** (subtype of Doctor): attribute: consultationFee

Relationships:

- **AssignedTo**: Ward (1,1) — (0,N) Patient
 - Each patient belongs to exactly one ward (total from Patient)
 - Each ward may have 0 or many patients (partial from Ward)
- **Treats**: Doctor (1,N) — (0,N) Patient
 - A doctor treats at least one patient (min-card=1 from Doctor)
 - A patient may be treated by many doctors (N-side)
 - Relationship attributes: treatmentDate, diagnosis
- **Generalization**: Doctor → Surgeon / Physician
 - Disjoint (a doctor is either surgeon or physician, not both)
 - Total (every doctor must be one of the two subtypes)

Key constraints to show:

- phone marked with { } (multi-valued)
- age marked with () (derived)
- Treats is N:M with attributes on the relationship diamond
- Subtype: triangle symbol + disjoint/total annotation

QUESTION 2 — E-R to Relational Schema Reduction (Solution)

```
Member(memberId, name, address, phone)
  PK: memberId
```

```
MemberMobile(memberId, mobile)
  PK: (memberId, mobile)
  FK: memberId REFERENCES Member(memberId) ON DELETE CASCADE
```

```
Book(bookId, title, publisher, year)
  PK: bookId
```

```
Author(authorId, name)
  PK: authorId
```

```
Borrows(memberId, bookId, dateDue)
  PK: (memberId, bookId) ← memberId chosen as PK since each member borrows
0-3 books (max-card=3 on Book side), but bookId alone can't be PK since each
book may be borrowed by many members (N:M in nature)
  FK: memberId REFERENCES Member(memberId) ON DELETE CASCADE
  FK: bookId REFERENCES Book(bookId) ON DELETE CASCADE
```

```
WrittenBy(bookId, authorId)
  PK: bookId ← because Book has (1,1) participation: written by exactly
one author
```

Wait – re-reading: Book connectivity is (1,N) and Author is (1,1).

Book-to-Author is N:1 (each book has exactly 1 author; each author writes 1+ books).

Actually: Book –(1,N)– WrittenBy –(1,1)– Author means:

- Each book must be written by exactly one author
- Each author has written at least one book

So it's an N:1 relationship. We can MERGE WrittenBy into

Book:

```
Correct (merged, minimized):
Book(bookId, title, publisher, year, authorId)
  PK: bookId
  FK: authorId REFERENCES Author(authorId) ON DELETE CASCADE
  -- authorId NOT NULL (total participation from Book)
```

WrittenBy is eliminated (merged into Book since N:1 from Book to Author).

Corrected, minimized solution:

```
Member(memberId, name, address, phone)
```

```
  PK: memberId
```

```
MemberMobile(memberId, mobile)
```

```
  PK: (memberId, mobile)
```

```
  FK: memberId REFERENCES Member(memberId) ON DELETE CASCADE
```

```
Book(bookId, title, publisher, year, authorId)
```

```
  PK: bookId
```

```
  FK: authorId REFERENCES Author(authorId) ON DELETE SET NULL [partial from  
Book? No – (1,1) total!]
```

```
  -- Correction: ON DELETE CASCADE or NO ACTION (authorId is NOT NULL since  
participation is total)
```

```
  -- Actually: since Book has total participation (1,1) in WrittenBy,  
authorId must be NOT NULL.
```

```
  -- ON DELETE: if a Book is deleted, nothing happens to Author. If an  
Author is deleted
```

```
  -- and the Book still exists... this violates total participation. So we  
should use
```

```
  -- ON DELETE CASCADE (delete the book if its author is deleted) or  
RESTRICT (forbid deleting author).
```

```
Author(authorId, name)
```

```
  PK: authorId
```

```
Borrows(memberId, bookId, dateDue)
```

```
  PK: (memberId, bookId)  -- each (member, book) pair borrowed at most once  
at any given time
```

```
  Alternative design: PK: (bookId, memberId, dateDue) if a member can borrow  
the same book multiple times
```

```
  FK: memberId REFERENCES Member(memberId) ON DELETE CASCADE
```

```
  FK: bookId REFERENCES Book(bookId) ON DELETE CASCADE
```

QUESTION 3 — Reverse Engineering (Solution)

3.1 CREATE TABLE Order

Create Order:

1. Department (no FKs)

But wait – Department has mgrEid FK → Employee. So Employee should come

first.

But Employee has deptCode FK → Department. Circular dependency!

Resolution:

Step 1: CREATE TABLE Employee(eid, name, salary, deptCode, ...) -- defer deptCode FK

Step 2: CREATE TABLE Department(deptCode, deptName, location, mgrEid, FOREIGN KEY (mgrEid) REFERENCES Employee(eid))

Step 3: ALTER TABLE Employee ADD FOREIGN KEY (deptCode) REFERENCES Department(deptCode)

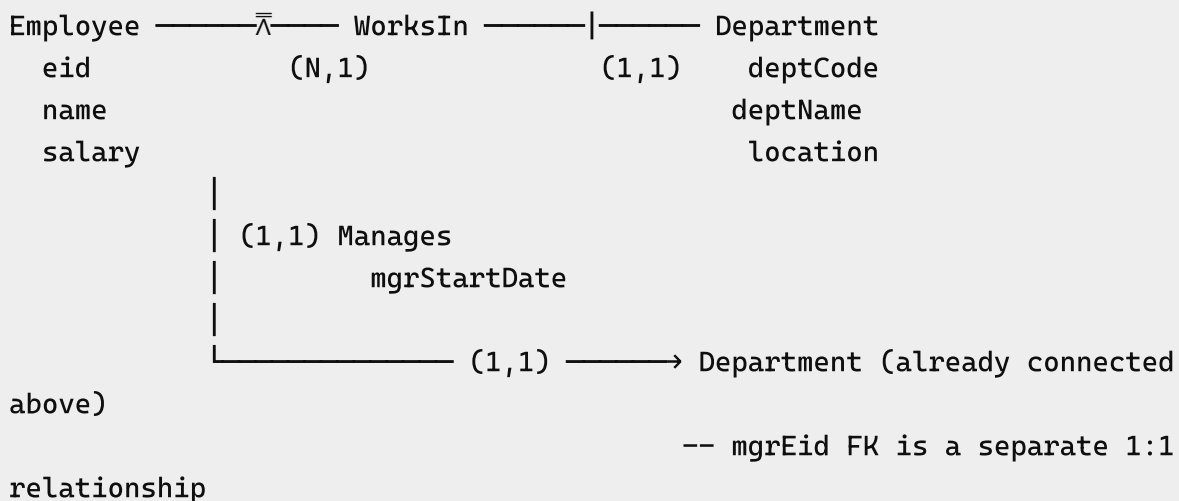
Step 4: CREATE TABLE Project(projNo, title, budget, deptCode, FOREIGN KEY (deptCode) REFERENCES Department(deptCode))

Step 5: CREATE TABLE WorksOn(eid, projNo, hours, FOREIGN KEY (eid) REFERENCES Employee(eid), FOREIGN KEY (projNo) REFERENCES Project(projNo))

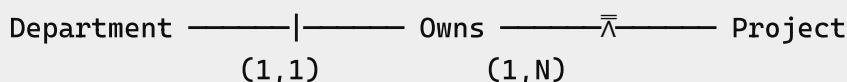
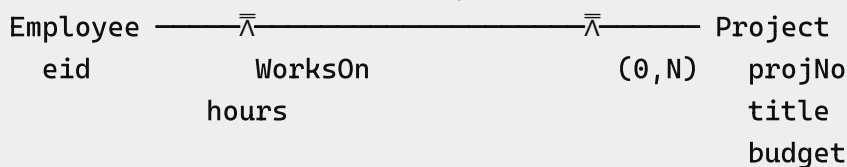
Step 6: CREATE TABLE Dependent(eid, depName, birthDate, relationship, FOREIGN KEY (eid) REFERENCES Employee(eid) ON DELETE CASCADE)

Circular FK dependency between Employee and Department is resolved by creating one table first without the FK, then adding it via ALTER TABLE afterward.

3.2 E-R Schema



Note: Manages is a separate 1:1 relationship from Employee to Department. (0,N)



(1,1)	(0,N)	(eid, depName) birthDate relationship
-------	-------	---

QUESTION 4 — FDs & Normalization (Solution)

Given: $R(A, B, C, D, E, F)$ with $F = \{ AB \rightarrow C, C \rightarrow D, AB \rightarrow E, E \rightarrow F, D \rightarrow A \}$

4.1 $\{A, B\}^+$

```

Step 0: {A, B}
Step 1: AB → C  ⇒ {A, B, C}
Step 2: C → D   ⇒ {A, B, C, D}
Step 3: AB → E  ⇒ {A, B, C, D, E}
Step 4: E → F   ⇒ {A, B, C, D, E, F}
Step 5: No new attributes added.

{A, B}^+ = {A, B, C, D, E, F} = all attributes of R

```

4.2 Candidate Keys

- a) **AB**: $AB^+ = \{A, B, C, D, E, F\} = R$. AB is a superkey. Is it minimal? $A^+ = \{A\}$ (none of the FDs have A alone on LHS, except indirectly via $D \rightarrow A$ but that gives A only). $B^+ = \{B\}$. Neither A nor B alone is a superkey, so AB is minimal → **AB is a candidate key**.
- b) **ABC**: $ABC^+ = R$ (since AB is already a superkey). But ABC is not minimal (C can be removed since $AB \rightarrow C$). **Not a candidate key** (not minimal).
- c) **ABE**: Same as above — AB alone is a superkey, so adding E makes it non-minimal. **Not a candidate key**.
- d) **ABD**: Same reasoning — not minimal. **Not a candidate key**.
- e) **The answer is (a) — only AB.**

Let's also check if there are other candidate keys:

- DB^+ : $D \rightarrow A$ gives A, then AB gives everything. $DB^+ = \{D, B, A, C, E, F\} = R$. DB is a superkey. Is it minimal? $D^+ = \{D, A\} \neq R$. $B^+ = \{B\} \neq R$. Yes, DB is minimal → **DB is also a candidate key**.

- EB^+ : $E \rightarrow F$, then? E alone: $E^+ = \{E, F\}$. $EB^+ = \{E, B, F\}$... Can we get to C? We need AB for $AB \rightarrow C$. We have B but need A. $D \rightarrow A$ but we need D. $C \rightarrow D$ but we need C. Dead end. $EB^+ = \{E, B, F\} \neq R$. **EB is not a superkey.**

So candidate keys are: **AB** and **DB**.

The question asks "Which of the following" — only AB is listed. But for completeness, DB is also a candidate key.

Answer: (a) AB

4.3 Prime Attributes

Prime attributes = attributes that belong to at least one candidate key.

Candidate keys: AB, DB

Prime attributes = {A, B, D}

Non-prime attributes = {C, E, F}

4.4 Is R in 3NF?

Check each FD in F against the 3NF condition:

FD	LHS is superkey?	RHS is prime?	3NF satisfied?
$AB \rightarrow C$	Yes (AB is CK)	C is non-prime	✓ Yes (superkey)
$C \rightarrow D$	No	D is prime	✓ Yes (D is prime)
$AB \rightarrow E$	Yes (AB is CK)	E is non-prime	✓ Yes (superkey)
$E \rightarrow F$	No	F is non-prime	✗ VIOLATION
$D \rightarrow A$	No	A is prime	✓ Yes (A is prime)

$E \rightarrow F$ violates 3NF because E is not a superkey, F is not a prime attribute, and the FD is non-trivial.

4.5 3NF Decomposition (Synthesis Algorithm)

Step 1: Compute canonical cover F_c

$F = \{AB \rightarrow C, C \rightarrow D, AB \rightarrow E, E \rightarrow F, D \rightarrow A\}$

Check for extraneous attributes:

- $AB \rightarrow C$: A from LHS? B^+ under $F = \{B\}$. $C \notin B^+$. Not extraneous.
- $AB \rightarrow C$: B from LHS? $A^+ = \{A\}$. $C \notin A^+$. Not extraneous.
- $AB \rightarrow E$: A from LHS? Same check. Not extraneous.
- $AB \rightarrow E$: B from LHS? Same check. Not extraneous.

Combine $AB \rightarrow C$ and $AB \rightarrow E$: $AB \rightarrow CE$ (by union rule).

$F_c = \{ AB \rightarrow CE, C \rightarrow D, E \rightarrow F, D \rightarrow A \}$

Check for redundant FDs:

- Is $AB \rightarrow CE$ derivable from others? Remove it. AB^+ (using $\{C \rightarrow D, E \rightarrow F, D \rightarrow A\}$) = $\{A, B\}$. Not derivable. Keep.
- $C \rightarrow D$? Remove. $C^+ = \{C\}$. Not derivable. Keep.
- $E \rightarrow F$? Remove. $E^+ = \{E\}$. Not derivable. Keep.
- $D \rightarrow A$? Remove. $D^+ = \{D\}$. Not derivable. Keep.

$F_c = \{ AB \rightarrow CE, C \rightarrow D, E \rightarrow F, D \rightarrow A \}$

Step 2: Create relations from F_c

$R_1(A, B, C, E)$ — from $AB \rightarrow CE$

$R_2(C, D)$ — from $C \rightarrow D$

$R_3(E, F)$ — from $E \rightarrow F$

$R_4(D, A)$ — from $D \rightarrow A$

Step 3: Check if any relation contains a candidate key

Candidate keys: AB, DB

- R_1 : Contains AB  (covers CK AB)
- R_2 : C only — does not contain AB or DB
- R_3 : E, F — does not contain AB or DB
- R_4 : D, A — does not contain AB or DB

R_1 contains AB, which is a candidate key. So no additional relation is needed.

Final 3NF decomposition:

- $R1(A, B, C, E) \text{ — CK: } AB$
- $R2(C, D) \text{ — CK: } C$
- $R3(E, F) \text{ — CK: } E$
- $R4(D, A) \text{ — CK: } D$

Properties:

- ☒ Lossless join: $R1 \cap R2 = \{C\}$, and $C \rightarrow D$ means C is key of $R2$. $R1 \cap R3 = \{E\}$, and $E \rightarrow F$ means E is key of $R3$...
 - Actually let's verify more carefully. The synthesis algorithm always produces a lossless decomposition when we add a candidate key relation. Since AB is in $R1$, the decomposition is lossless.
- ☒ Dependency preserving: each FD from F_c is entirely within one relation schema.

4.6 Is R in BCNF?

Check each FD against the BCNF condition (LHS must be a superkey):

FD	LHS is superkey?	BCNF satisfied?
$AB \rightarrow C$	Yes	☒
$C \rightarrow D$	$C^+ = \{C, D, A, \dots\}$ — C alone gives $\{C, D, A\}$. Not all of R. No	✗ VIOLATION
$AB \rightarrow E$	Yes	☒
$E \rightarrow F$	$E^+ = \{E, F\}$. Not all of R. No	✗ VIOLATION
$D \rightarrow A$	$D^+ = \{D, A\}$. Not all of R. No	✗ VIOLATION

No, R is not in BCNF. Violating FDs: $C \rightarrow D$, $E \rightarrow F$, $D \rightarrow A$.

4.7 BCNF Decomposition

Start: $S = \{R(A, B, C, D, E, F)\}$

Iteration 1: R violates BCNF via $C \rightarrow D$ (C is not a superkey).

- $R - D = R(A, B, C, E, F) \rightarrow R1(A, B, C, E, F)$
- $(C, D) \rightarrow R2(C, D)$
- $S = \{R1(A, B, C, E, F), R2(C, D)\}$

Iteration 2: Check R1(A, B, C, E, F). FDs that apply to R1: $AB \rightarrow C$, $AB \rightarrow E$, $C \rightarrow \dots$ ($C \rightarrow D$ is in R2, not applicable to R1 with C only), $AB \rightarrow \dots$, $E \rightarrow F$.

Calculate CK of R1: $AB^+ = \{A, B, C, E, F\}$. AB is CK.

$E \rightarrow F$ in R1? E^+ within R1 = $\{E, F\}$. E is not a superkey for R1. **Violation!**

- $R1 - F = R1a(A, B, C, E)$
- $(E, F) \rightarrow R3(E, F)$
- $S = \{R1a(A, B, C, E), R2(C, D), R3(E, F)\}$

Iteration 3: Check R1a(A, B, C, E). FDs: $AB \rightarrow C$, $AB \rightarrow E$. Also $D \rightarrow A$ is from R4 — does it apply here? R1a doesn't have D, so no. What about $C \rightarrow \dots$ C alone? C^+ in R1a = $\{C\}$. C is not a superkey for R1a, but what FD with C as LHS applies to R1a? None ($C \rightarrow D$ had D, which is now in R2). So for R1a, the FDs that hold are $AB \rightarrow CE$. CK = AB. AB is a superkey for all FDs. **BCNF satisfied.**

Iteration 4: Check R2(C, D). FD: $C \rightarrow D$. C is a superkey (and CK). **BCNF satisfied.**

Iteration 5: Check R3(E, F). FD: $E \rightarrow F$. E is a superkey (and CK). **BCNF satisfied.**

Final BCNF decomposition:

- $R1a(A, B, C, E)$ — CK: AB
- $R2(C, D)$ — CK: C
- $R3(E, F)$ — CK: E

FDs preserved: $AB \rightarrow CE$ ✓, $C \rightarrow D$ ✓, $E \rightarrow F$ ✓

FD lost: $D \rightarrow A$ ✗ (requires joining R1a with R2 on C, but $D \rightarrow A$ spans R2 and would need R1a which doesn't contain D—actually, to check $D \rightarrow A$: D is in R2, A is in R1a. These two relations don't have a common attribute that would allow checking the FD. Wait — R1a has C and R2 has C. $D \rightarrow A$: $R2(D, C) \bowtie R1a(A, B, C, E)$. The join of these two on C gives us A and D in one relation. But can we verify $D \rightarrow A$ without joining? No — $D \rightarrow A$ is **lost**.)

4.8 3NF vs BCNF — Recommendation

Recommend 3NF. The 3NF decomposition $\{R1(A, B, C, E), R2(C, D), R3(E, F), R4(D, A)\}$ preserves all FDs, while the BCNF decomposition loses $D \rightarrow A$. Losing this FD means the database cannot enforce the constraint that each D value determines a unique A value without expensive application-level JOIN checks or triggers. In practice, 3NF is often "good enough" and the redundancy introduced by $D \rightarrow A$ in a separate relation is minimal.

QUESTION 5 — Relational Algebra (Solution)

5.1

```
π Student.name (
  σ title='Database Systems' (
    Course ⋈ Enrol ⋈ Student
  )
)
```

Or more explicitly:

```
π name (
  Student ⋈ (
    Enrol ⋈ (
      σ title='Database Systems' (Course)
    )
  )
)
```

5.2

```
π name (
  Instructor ⋈ Teaching ⋈ Course ⋈ σ deptName='Computer Science'
  (Department)
)
-
π name (
  Instructor ⋈ Teaching ⋈ Course ⋈ σ deptName='Mathematics' (Department)
)
```

5.3

The SQL does a GROUP BY deptName with HAVING COUNT(*) >= 5. In relational algebra, we need to find departments where at least 5 distinct student pairs exist.

Approach: Find department-student combinations, then for each department, find those where there exist at least 5 different students.

```
π deptName (
  Student ⋈ S1
)
```

```

)
where S1 is a self-reference...

A cleaner approach using renaming:
π deptName (
  σ S1.studentId ≠ S2.studentId ∧ S1.deptName = S2.deptName (
    ρ S1(Student) × ρ S2(Student)
  )
)

```

Wait — that gives departments with at least 2 students, not 5. For at least 5, we need a more complex expression or acknowledge that basic RA cannot directly express arbitrary counting.

The intended solution pattern (from the original review exercises, Q5.3):

```

π P1.deptName (
  σ P1.deptName = P2.deptName = P3.deptName = P4.deptName = P5.deptName
    ∧ P1.studentId ≠ P2.studentId ≠ P3.studentId ≠ P4.studentId ≠
    P5.studentId (
      ρ P1(Student) × ρ P2(Student) × ρ P3(Student) × ρ P4(Student) × ρ
      P5(Student)
    )
)

```

Each P_i is a renamed copy of Student. The join of 5 copies with the condition that all deptNames are equal and all studentIds are different finds departments with at least 5 distinct students. Then we project deptName.

5.4

```

π name, salary (
  Instructor ⋈ (
    σ salary > MAX_History (
      ρ MAX_History(π MAX(salary) (σ deptName='History' (Instructor)))
    )
  )
)

```

Wait — relational algebra doesn't have MAX as a basic operator. We need:

```

π I.name, I.salary (
  ρ I(Instructor) ⋈
  σ I.salary > H.salary (
    ρ I(Instructor) × ρ H(σ deptName='History' (Instructor))
  )
)

```

But this gives instructors who earn more than *some* History instructor, not *every* History instructor. For "more than every," we use:

```

π name, salary (
  Instructor
)
-
π I.name, I.salary (
  ρ I(Instructor) ⋈
  σ I.salary ≤ H.salary (
    ρ I(Instructor) × ρ H(σ deptName='History' (Instructor))
  )
)

```

Subtract those who earn $\leq$ some History instructor from all instructors, leaving those who earn more than ALL History instructors.

QUESTION 6 — SQL Queries (Solution)

6.1

```

SELECT DISTINCT C.clientId, C.name
FROM Client C, Depositor D, Account A, Borrower B, Loan L
WHERE C.clientId = D.clientId
  AND D.accountNo = A.accountNo
  AND C.clientId = B.clientId
  AND B.loanNo = L.loanNo
  AND A.branchName = 'Star House'
  AND L.branchName = 'Star House';

```

Alternate (using NATURAL JOIN — but careful with common attributes):

```

SELECT DISTINCT clientId, name
FROM Client

```

```

NATURAL JOIN Depositor
NATURAL JOIN Account
NATURAL JOIN Borrower
NATURAL JOIN Loan
WHERE branchName = 'Star House';

```

⚠️ **NATURAL JOIN** on all of these would join on all common attributes, which may produce incorrect results because **Account** and **Loan** both have **branchName** and **amount / balance**. The explicit JOIN...USING or WHERE-based join is safer here.

6.2

```

SELECT branchName, COUNT(*) AS totalLoans
FROM Loan
GROUP BY branchName
HAVING COUNT(*) > 3
ORDER BY totalLoans DESC;

```

6.3

```

SELECT C.clientId, C.name
FROM Client C
WHERE NOT EXISTS (
    SELECT B.branchName
    FROM Branch B
    WHERE B.district = 'Yau Tsim Mong'
    MINUS
    SELECT A.branchName
    FROM Depositor D, Account A
    WHERE D.clientId = C.clientId
        AND D.accountNo = A.accountNo
);

```

Alternate using double NOT EXISTS:

```

SELECT C.clientId, C.name
FROM Client C
WHERE NOT EXISTS (
    SELECT *
    FROM Branch B
    WHERE B.district = 'Yau Tsim Mong'
        AND NOT EXISTS (
            SELECT *

```

```

        FROM Depositor D, Account A
        WHERE D.clientId = C.clientId
              AND D.accountNo = A.accountNo
              AND A.branchName = B.branchName
    )
);

```

6.4

```

WITH DeptMaxSal AS (
    SELECT deptName, MAX(salary) AS maxSal
    FROM Instructor
    GROUP BY deptName
)
SELECT I.deptName, I.name
FROM Instructor I, DeptMaxSal D
WHERE I.deptName = D.deptName
      AND I.salary = D.maxSal;

```

6.5

```

SELECT branchName, assets
FROM (
    SELECT branchName, assets,
           RANK() OVER (ORDER BY assets DESC) AS rnk
    FROM Branch
)
WHERE rnk <= 3;

```

Using RANK() rather than DENSE_RANK() : If branches have assets (500, 500, 400, 400, 300), RANK gives ranks (1, 1, 3, 3, 5) — returning 4 branches (two tied at #1, two tied at #3). With DENSE_RANK: (1, 1, 2, 2, 3) — returning the same 4 branches here. The choice depends on whether you want to fill exactly 3 "positions" (use DENSE_RANK) or 3 "rows max unless ties spill over" (use RANK). Both are acceptable with explanation.

6.6

```

SELECT D.deptName, D.budget,
       LISTAGG(S.name, ', ' ) WITHIN GROUP (ORDER BY S.name) AS studentList
FROM Department D, Student S
WHERE D.deptName = S.deptName

```

```
GROUP BY D.deptName, D.budget
HAVING COUNT(*) > 2;
```

6.7

```
SELECT branchName
FROM Branch
WHERE assets > (SELECT MAX(assets) FROM Branch WHERE district = 'Yau Tsim
Mong');
```

Since `> ALL (set)` is equivalent to `> MAX(set)`.

6.8

```
-- (1) INSERT
INSERT INTO Client (clientId, name, hkid, address, district, rating)
VALUES ('C-999', 'Chan Tai Man', 'A123456(7)', '1 Harbour Road', 'Wan Chai',
7);

-- (2) UPDATE with CASE
UPDATE Account
SET balance = CASE
    WHEN balance <= 10000 THEN balance * 1.06
    ELSE balance * 1.04
END
WHERE branchName = 'Pacific Place';

-- (3) DELETE
DELETE FROM Borrower
WHERE loanNo IN (
    SELECT loanNo
    FROM Loan
    WHERE branchName IN (
        SELECT branchName
        FROM Branch
        WHERE district = 'Yau Tsim Mong'
    )
);
```

QUESTION 7 — SQL DDL & PL/SQL (Solution)

7.1


```

CREATE TABLE Loan (
  loanNo      CHAR(5) PRIMARY KEY,
  amount      NUMBER(8,2) CHECK (amount BETWEEN 1000 AND 200000),
  year        CHAR(4) NOT NULL,
  branchName  VARCHAR2(15) NOT NULL,
  FOREIGN KEY (branchName) REFERENCES Branch(branchName)
    ON DELETE CASCADE
);

```

7.2

```

CREATE VIEW HighValueClients AS
SELECT clientId, name, district
FROM Client
WHERE rating >= 8;

-- Query using the view:
SELECT district, COUNT(*)
FROM HighValueClients
GROUP BY district;

```

7.3

```

CREATE OR REPLACE PROCEDURE TransferFunds (
  fromAccount IN Account.accountNo%TYPE,
  toAccount   IN Account.accountNo%TYPE,
  amount      IN NUMBER
) IS
  fromBal Account.balance%TYPE;
  toBal   Account.balance%TYPE;
  insufficient_funds EXCEPTION;
BEGIN
  -- Get balance of fromAccount
  SELECT balance INTO fromBal
  FROM Account
  WHERE accountNo = fromAccount
  FOR UPDATE; -- lock the row

  -- Check sufficient balance
  IF fromBal < amount THEN
    RAISE insufficient_funds;
  END IF;

  -- Get balance of toAccount (verify it exists)
  SELECT balance INTO toBal

```

```

FROM Account
WHERE accountNo = toAccount
FOR UPDATE;

-- Perform transfer
UPDATE Account SET balance = balance - amount WHERE accountNo =
fromAccount;
UPDATE Account SET balance = balance + amount WHERE accountNo = toAccount;

COMMIT;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: One or both accounts do not exist.');
```

```

WHEN insufficient_funds THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds in account ' ||
fromAccount);
WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END TransferFunds;
```

7.4

```

CREATE OR REPLACE TRIGGER update_branch_liabilities
AFTER INSERT ON Loan
FOR EACH ROW
BEGIN
    UPDATE Branch
    SET liabilities = liabilities + :NEW.amount
    WHERE branchName = :NEW.branchName;
END;
```

QUESTION 8 — Short Answer (Solution)

8.1 Logical vs Physical Data Independence

	Logical Data Independence	Physical Data Independence
Definition	Changes to the logical schema do not affect view schemas	Changes to the physical schema do not affect the logical schema
Example	Adding a new column to a base table does not break existing views that use <code>SELECT *</code> (views	Moving a table to a different disk or changing the index structure does

	Logical Data Independence	Physical Data Independence
	should be recompiled, but application queries against views still work)	not require rewriting SQL queries

8.2 RANK vs DENSE_RANK vs ROW_NUMBER

Given scores: (100, 90, 90, 80)

Score	RANK	DENSE_RANK	ROW_NUMBER
100	1	1	1
90	2	2	2
90	2	2	3
80	4	3	4

- **RANK**: Ties get same rank; next rank skips (1, 2, 2, 4) — has "gaps"
- **DENSE_RANK**: Ties get same rank; no gaps (1, 2, 2, 3)
- **ROW_NUMBER**: Every row gets a unique number, ties broken arbitrarily (1, 2, 3, 4)

8.3 3NF vs BCNF

Difference: BCNF is stricter — every FD's left-hand side must be a superkey. 3NF allows a violation if the right-hand side is a prime attribute.

Why choose 3NF? BCNF decompositions may not be dependency-preserving (some FDs span multiple relations). 3NF always guarantees a lossless-join, dependency-preserving decomposition. When enforcing FDs via application-level checks is expensive or error-prone, 3NF is preferred.

8.4 Aggregate vs Analytic Functions

Aggregate Functions	Analytic Functions
Collapse multiple rows into one row per group	Return one row per input row
Require GROUP BY when mixing with non-aggregate columns	Use PARTITION BY (optional, does not collapse)
Evaluated during GROUP BY phase	Evaluated last (before ORDER BY)

Usage: Use analytic functions when you need aggregate values alongside detail rows (e.g., "show each employee's salary AND the department average in the same row"). Use aggregate functions when you only need summary data.

8.5 WHERE vs HAVING

WHERE	HAVING
Filters individual rows before grouping	Filters groups after GROUP BY
Cannot contain aggregate functions	Can contain aggregate functions
Evaluated before GROUP BY	Evaluated after GROUP BY

Evaluation order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Why it matters: You cannot use **HAVING** to filter individual rows (it operates on groups). You cannot use **WHERE** to filter on aggregate results (aggregates don't exist yet when WHERE executes). For example:

```
-- ❌ Wrong: WHERE cannot use AVG
SELECT deptName, AVG(salary) FROM Instructor WHERE AVG(salary) > 50000 GROUP
BY deptName;

-- ✅ Correct
SELECT deptName, AVG(salary) FROM Instructor GROUP BY deptName HAVING
AVG(salary) > 50000;
```

END OF PRACTICE TEST